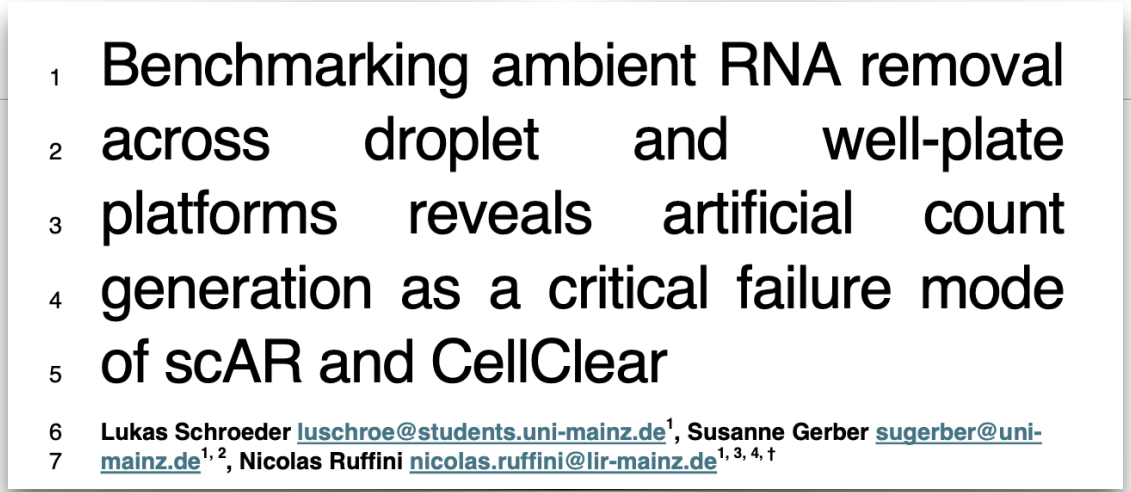
Exercise 7 notes

- Please use the template; sometimes it's surprisingly hard to find your answers but also graphics, table of contents, etc.

Project idea #1



1 Benchmarking ambient RNA removal
2 across droplet and well-plate
3 platforms reveals artificial count
4 generation as a critical failure mode
5 of scAR and CellClear

6 **Lukas Schroeder** luschroe@students.uni-mainz.de[1], **Susanne Gerber** sugerber@uni-
7 mainz.de[1, 2], **Nicolas Ruffini** nicolas.ruffini@lir-mainz.de[1, 3, 4, †]

- Reimplement some of their analyses into an Omnibenchmark
- Verify the “failure mode” (hypothesis: artefact)
- Add a detailed analysis

Project idea #2

On the illusion of scRNA-seq batch effect correction

Francesco Codicè [1], Piero Fariselli [1], Daniele Raimondi [2,*]

- Reimplement some of their analyses (in particular, simulations) into an Omnibenchmark
- Add a detailed analysis

RNA-seq workflow

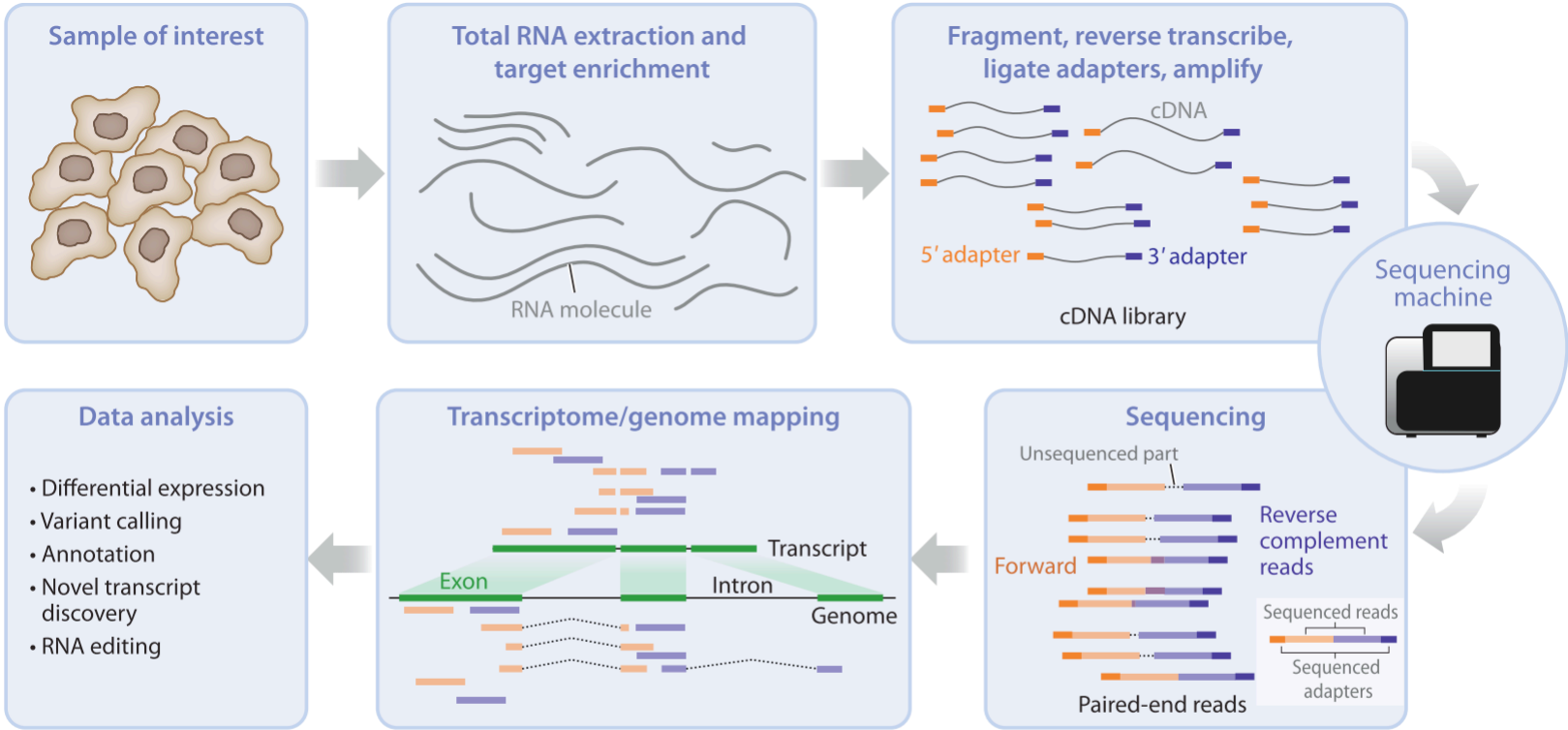


Figure 1

Overview of the experimental steps in an RNA sequencing (RNA-seq) protocol. The complementary DNA (cDNA) library is generated from isolated RNA targets and then sequenced, and the reads are mapped against a reference genome or transcriptome. Downstream data analysis depends on the goal of the experiment and can include, among other things, assessing differential expression, variant calling, or genome annotation.

Van den Berge, K., et al. (2019). RNA Sequencing Data: Hitchhiker's Guide to Expression Analysis. *Annual Review of Biomedical Data Science*

Ideally, you also (regarding of whether RNA-seq or another pipeline) ..

- Idea of workflows: automatic execution, not automatic data analysis
- Always important to perform "exploratory data analysis" at various points in a pipeline
- always check intermediate results and generate plots not just summaries! (e.g. FastQC)

ARMOR workflow

- **A**utomated = snakemake

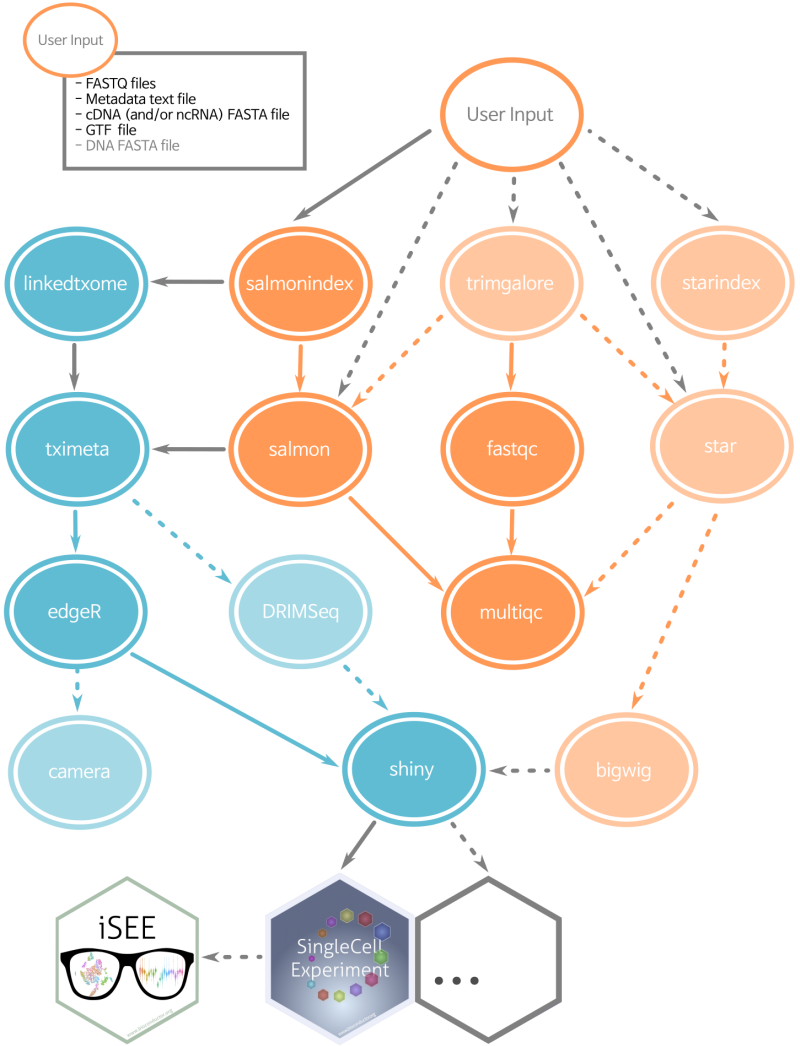
snakemake
A framework for reproducible data analysis

- **R**eproducible = conda and GitHub

CONDA GitHub Bioconductor
OPEN SOURCE SOFTWARE FOR BIOINFORMATICS

- **MO**dular = snakemake rules + configuration file

- **R**NA-seq



https://github.com/csoneson/ARMOR
Orjuela et al., G3 2019

Typical preprocessing of RNA-seq reads

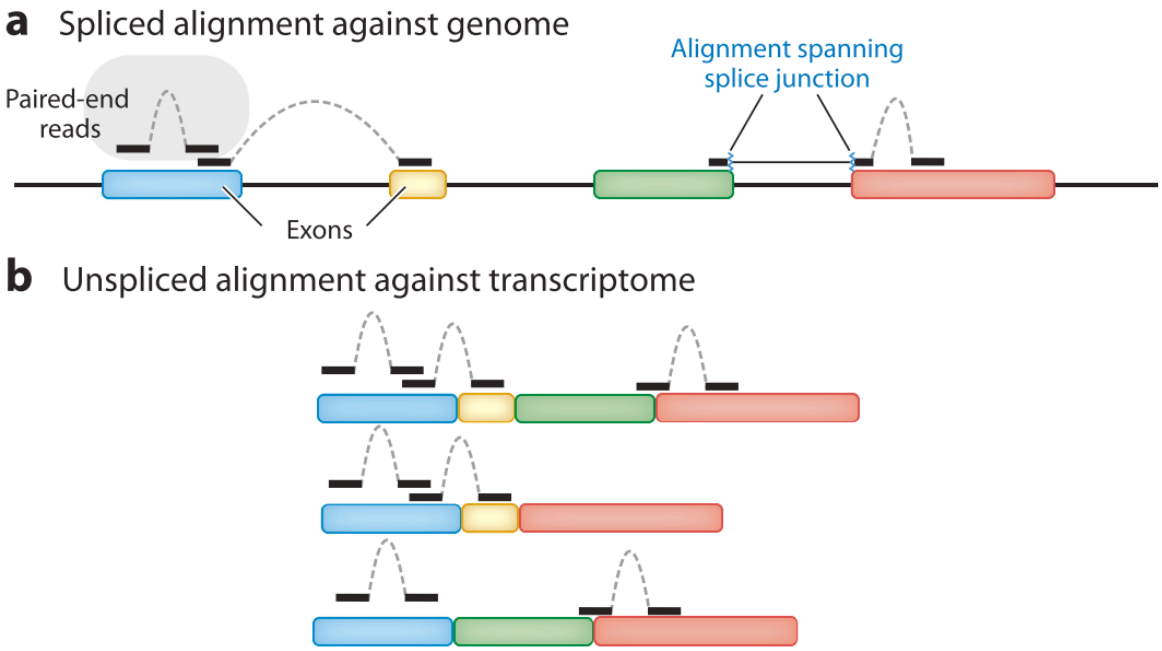
1. Quality filtering & adapter trimming
   (remove reads with bad quality & adapter sequences)
2. Alignment to reference genome
3. Quantification of feature of interest
   (gene or transcript)

Quality control

- **FastQC** https://www.bioinformatics.babraham.ac.uk/projects/fastqc/
- **MultiQC** https://multiqc.info/
  - aggregates FastQC results from multiple samples, as well as Salmon and STAR output
- # reads, read length, read quality, GC content, % duplicated reads, adapter contamination, …
- Tools for quality filtering/adapter trimming:

  cutadapt, TrimGalore!, Trimmomatic, FASTX-toolkit, …

Alignment



STAR https://github.com/alexdobin/STAR

HISAT2 http://ccb.jhu.edu/software/hisat2/index.shtml

Salmon https://combine-lab.github.io/salmon/about/

kallisto https://pachterlab.github.io/kallisto/about

Van den Berge, K., et al. (2019). RNA Sequencing Data: Hitchhiker's Guide to Expression Analysis. *Annual Review of Biomedical Data Science*

Statistical analysis

- **Differential gene expression:** Which genes change in expression in different genotypes, treatments, time points, …?

(edgeR http://bioconductor.org/packages/release/bioc/html/edgeR.html or DEseq2 http://bioconductor.org/packages/release/bioc/html/DESeq2.html or limma-voom, etc.)

- **Differential transcript usage**: Does the transcript composition of a given gene change?

(DRIMseq https://bioconductor.org/packages/release/bioc/html/DRIMSeq.html)

- **Gene set analysis**: are the DE genes enriched for a specific gene annotation category?

(`camera()` function from limma R package https://academic.oup.com/nar/article/40/17/e133/2411151)

Managing software environments: many options

- Linux, Mac, Windows
- R has a packaging system, Python has a packaging system, but also many tools you either can download an executable version or the source code to compile into an executable
- Important to be conscious of your “current state”: PATH —> versions and environments that you are running; often with multi-package software environments, only certain versions
- install everything "locally" (manually) versus software environments (e.g., conda) versus containers (e.g., docker, singularity)
- no single correct way to manage software, but important to know that many options exist; to think about how you manage software in projects that you will run (e.g., over time when versions change)

Docker




```
# list set of local images
docker image ls

# retrieve a docker image from dockerhub
docker pull markrobinsonuzh/sta426hs2025:latest

# if you build an image locally
# docker build
# docker push

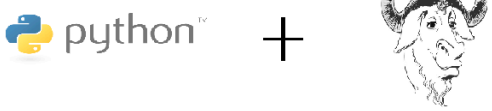
# running containers
docker run --platform=linux/amd64 \
  -v /Users/mark/scratch:/home/rstudio/mnt -e \
  PASSWORD=bioc --cpus 4 --memory 8GB -p 8888:8787    \
  markrobinsonuzh/sta426hs2025:latest

docker run -it markrobinsonuzh/docker-for-sta426hs2025:latest /bin/bash

# list running containers
docker ps

# login to running container
docker exec -it f9f60233082f /bin/bash
```

Snakemake



https://snakemake.readthedocs.io/en/stable/

- workflow management system

→ reproducible and scalable data analyses

- specify **rules** that describe *how to create output files from input files*
- file/rule dependencies are automatically determined
- rules can use shell commands, python code or external python/R scripts
- runs on laptops, clusters, the cloud without modifications
- you can deploy software with conda (many other options)

What does it look like?

- Define Snakefile with rules

```
rule hello:
    input:
        "my_name.txt"
    output:
        "hello.txt"
    shell:
        "NAME=$(cat {input}); "
        "echo Hello $NAME! > {output}"
```

- Execute with

```
(snakemake) katharina@IMLS-NBM-KHE:~/Desktop$ snakemake hello --cores 1
```

Snakemake: useful commands

`--help` to get detailed help message

`--use-conda` to run rules in conda environments

`-n` dry run → only display what would be done but do not execute anything

`-p` print shell commands that will be executed

`-r` print reason for each executed rule

can be combined in `-npr`

`-l` list all available rules

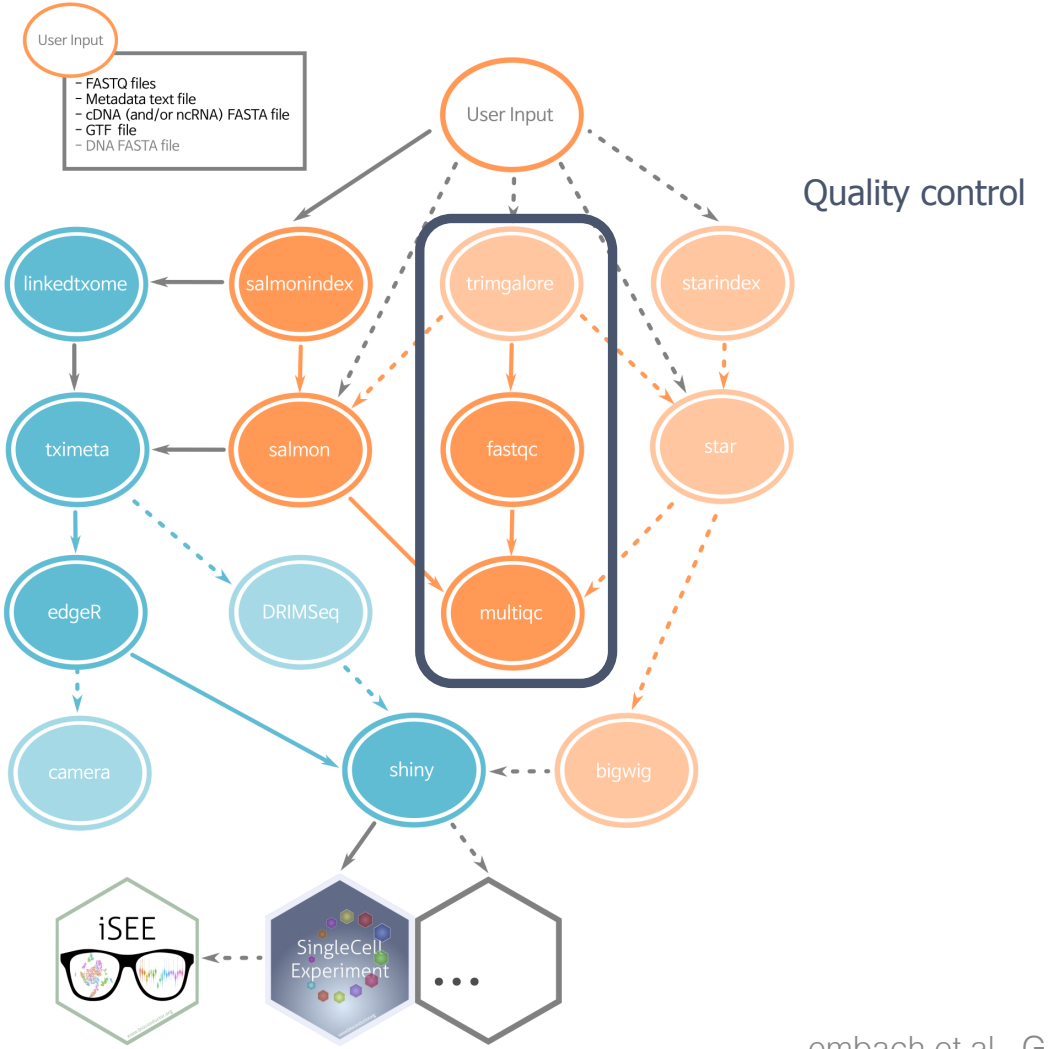
--cores to use at most this number of cores in parallel

--configfile path to configuration file (e.g. config.yaml)

ARMOR WORKFLOW

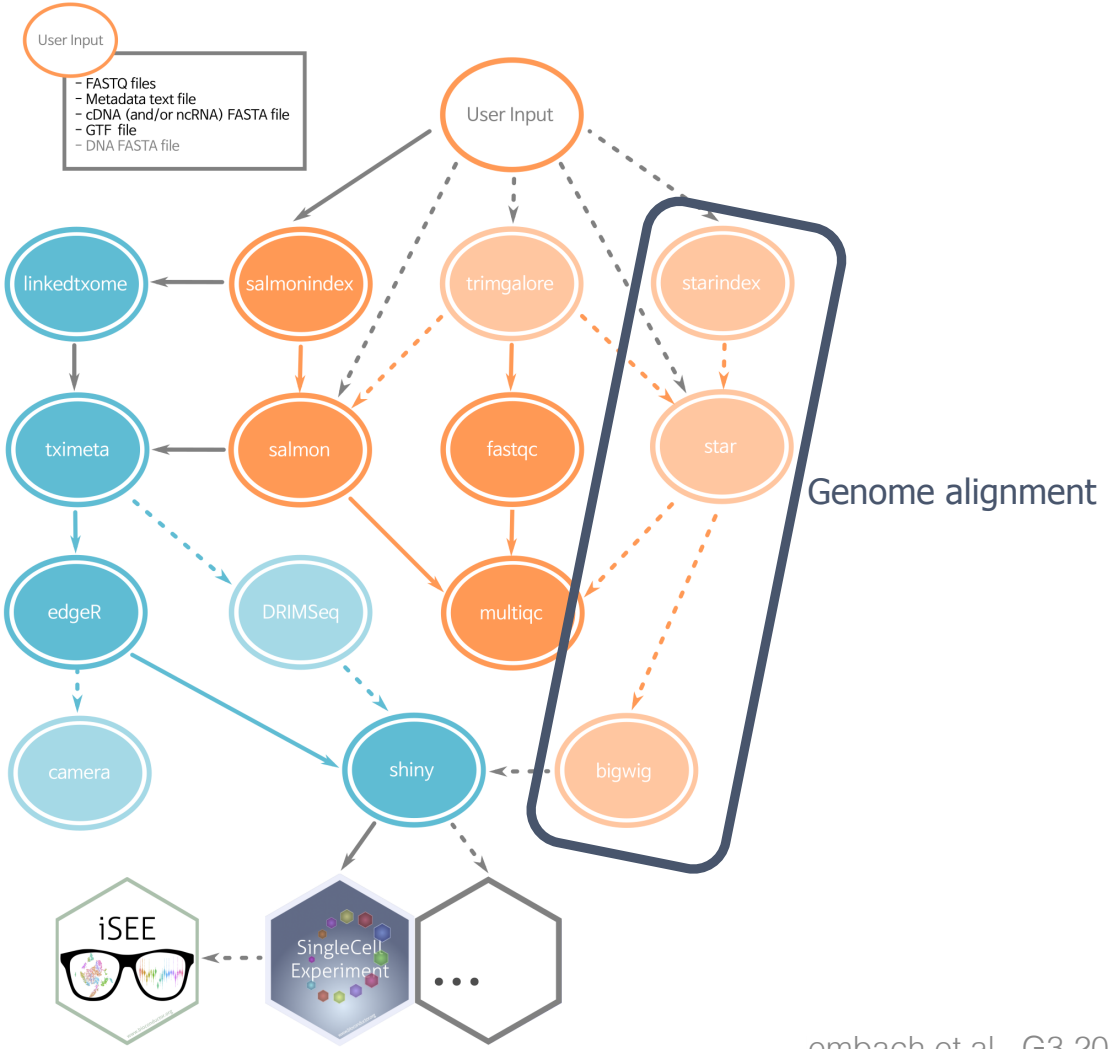


ARMOR workflow



Srijuela, Huang, Frembach et al., G3 2019

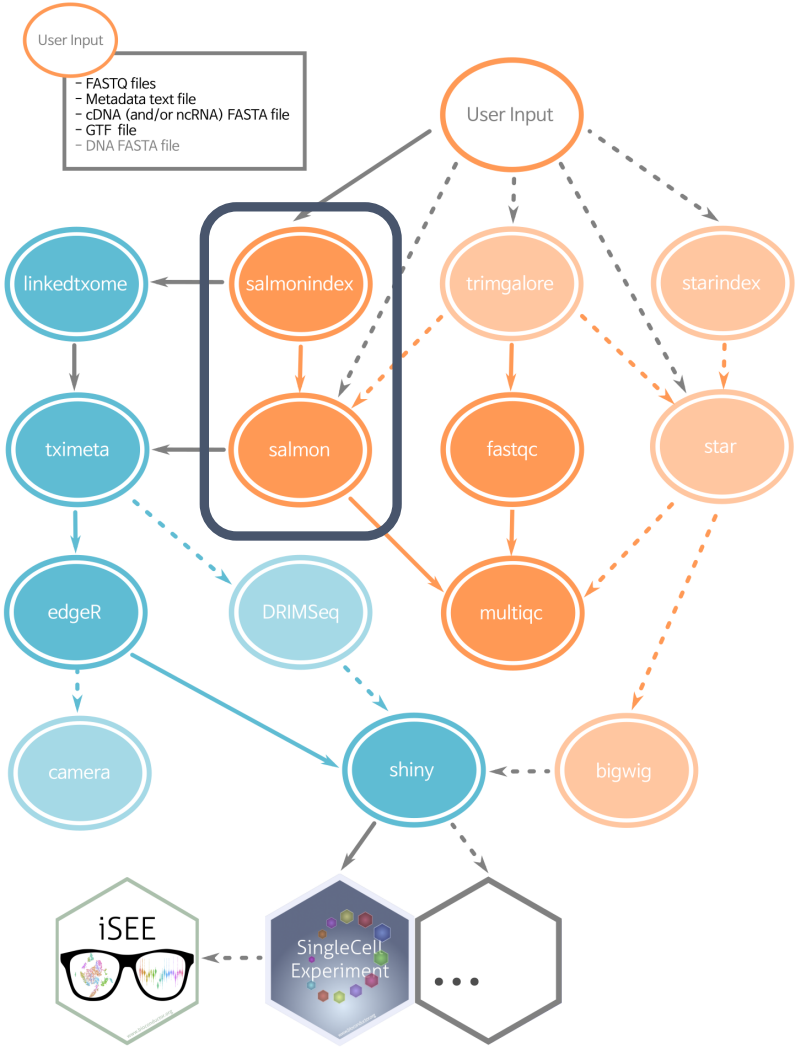
ARMOR workflow



Srijuela, Huang, Rembach et al., G3 2019

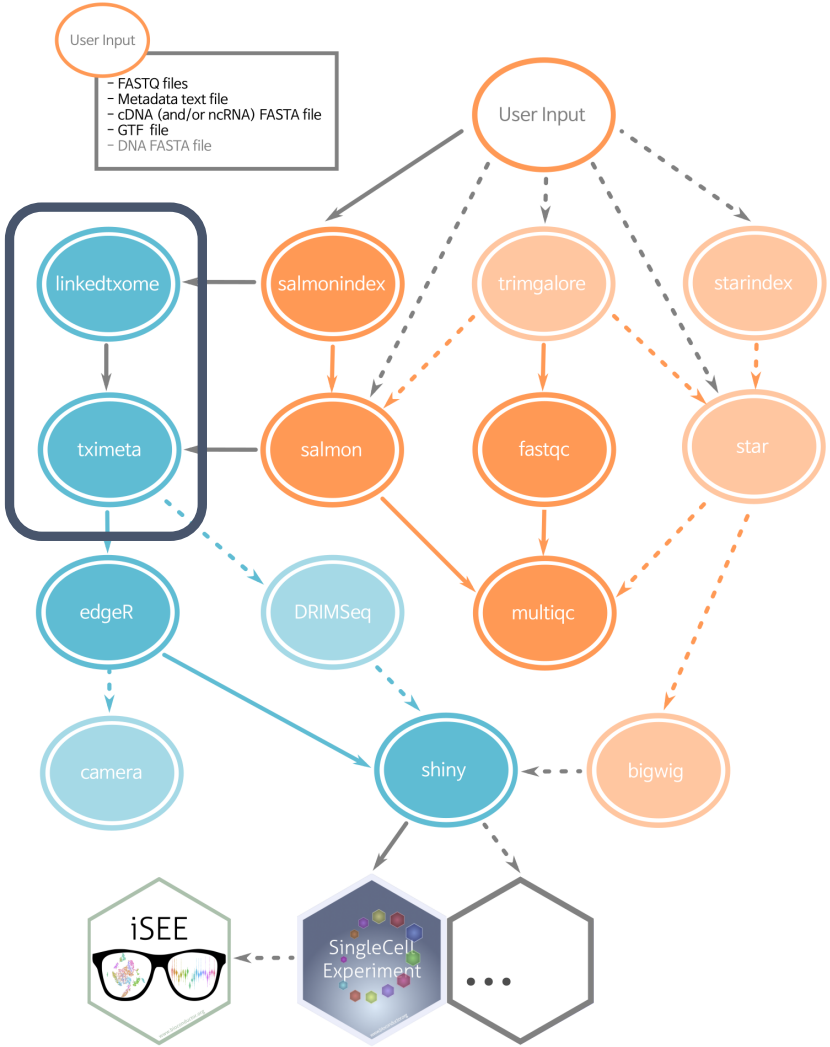
ARMOR workflow

Transcript abundance estimation



ARMOR workflow

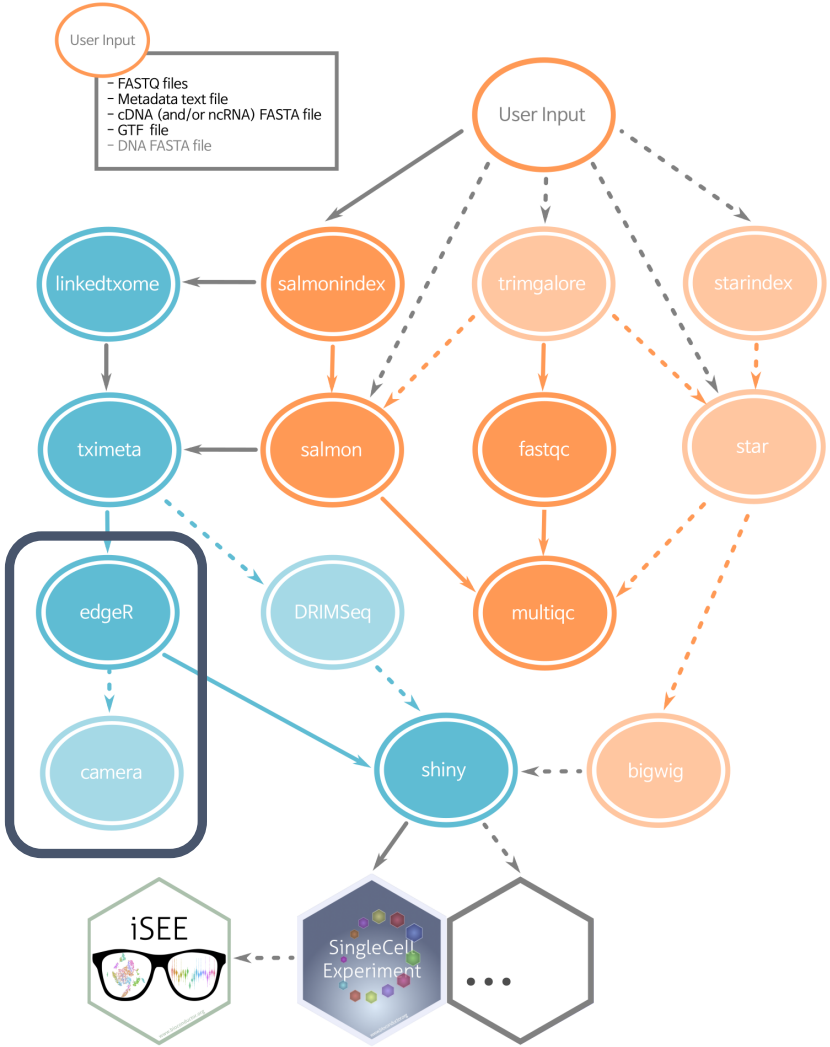
Data import into R



Srijuela, Huang, Rembach et al., G3 2019

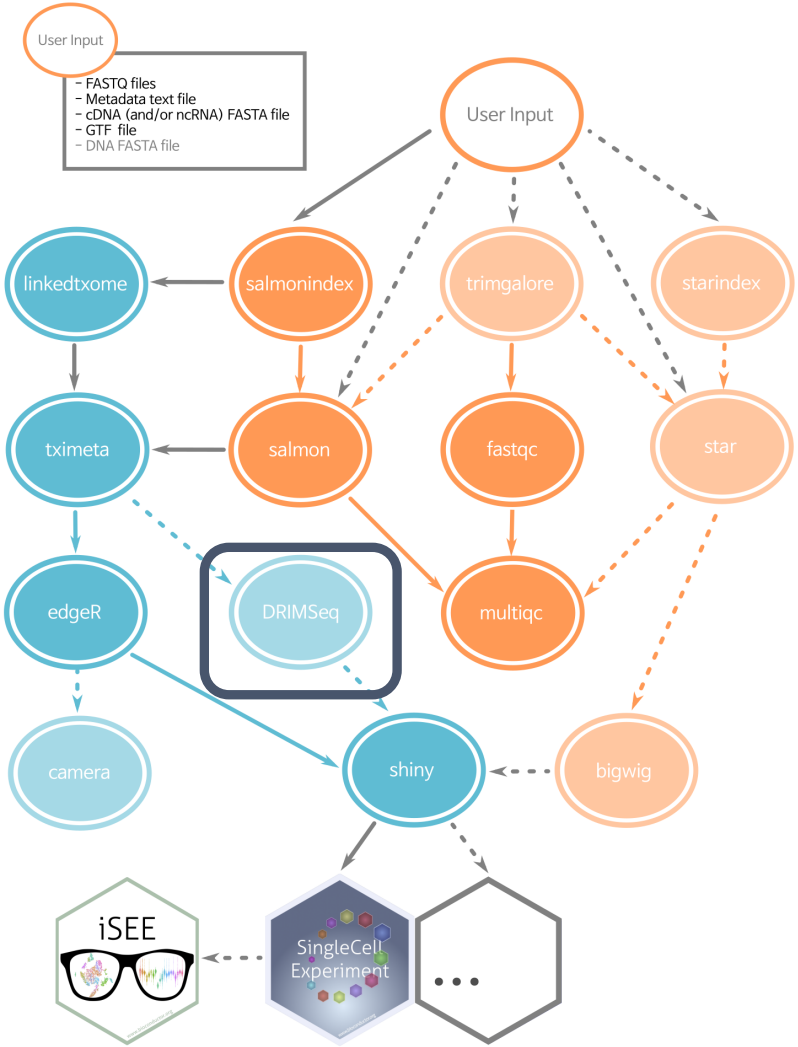
ARMOR workflow

Differential gene expression analysis & gene set enrichment

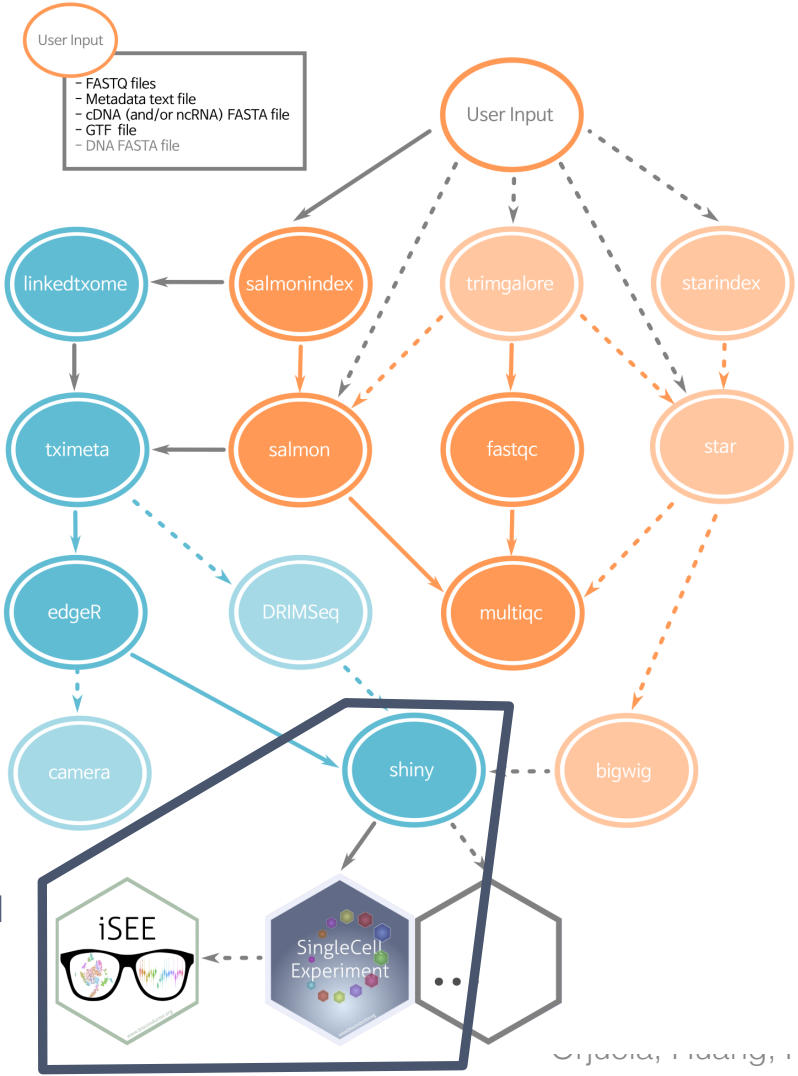


ARMOR workflow

Differential transcript usage



ARMOR workflow

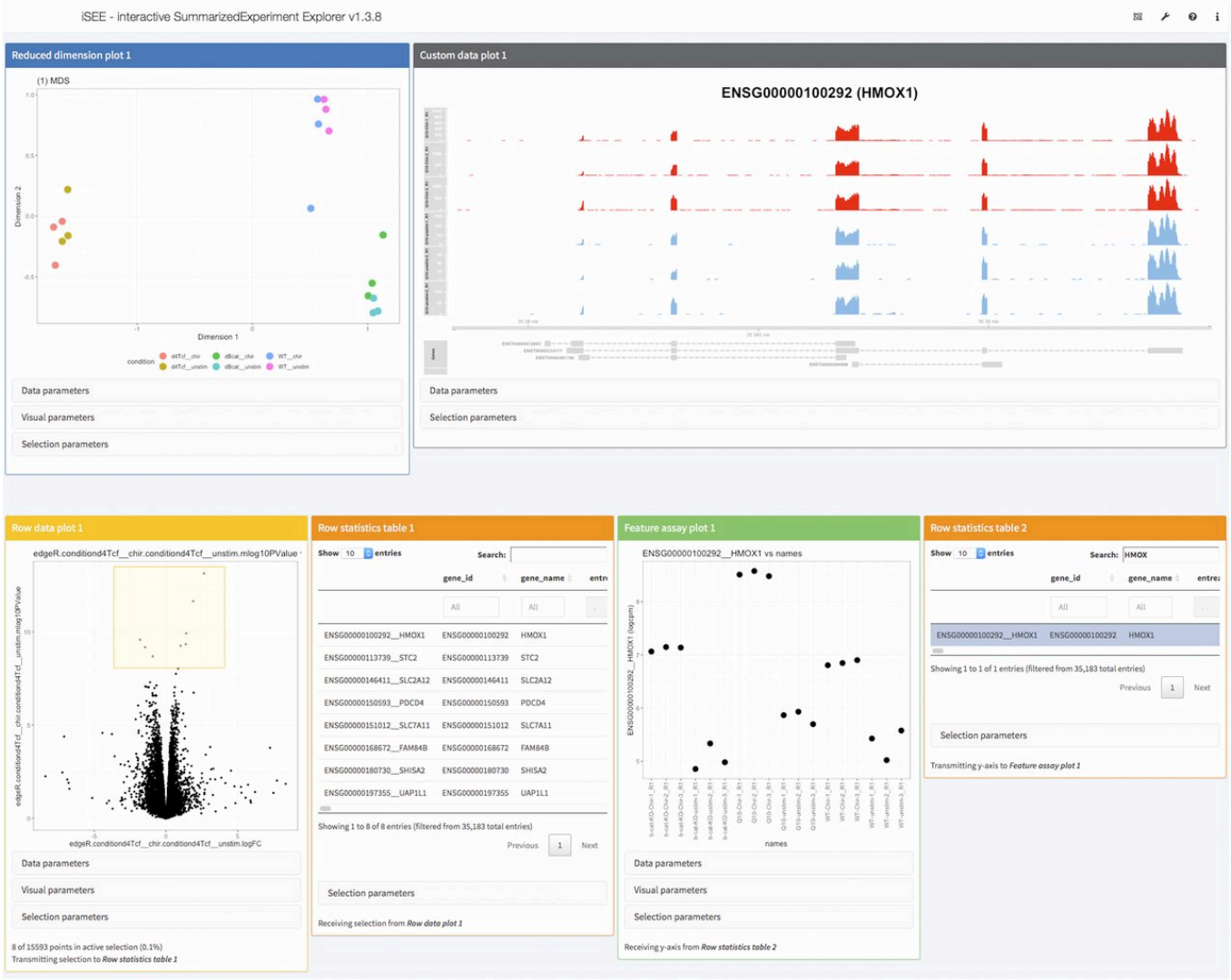


Data export and visualisation

ARMOR



- **A**utomated **R**eproducible **MO**dular **R**NA-seq
- https://github.com/csoneson/ARMOR
- https://www.g3journal.org/content/9/7/2089
- Snakemake workflow
- reproducible, automated, partially contained
- mix of command line tools and R
- Snakefile, configuration file and R scripts
- all software can be installed in conda environments
- visualization with iSEE R package → shiny app
- can be extended by adding rules to the Snakefile



iSEE visualization of the ARMOR output

https://bioconductor.org/packages/release/bioc/html/iSEE.html

Read coverage in integrative genome viewer (IGV)

- Visualize BAM/bigwig files and gene annotations (GTF) in IGV

- Start IGV and load files (BAM needs index)

ARMOR: useful commands

- snakemake –npr to see what snakemake will be executing
- snakemake setup to see if all required software is available
- snakemake checkinputs to see if your specified design and contrast matrix is valid